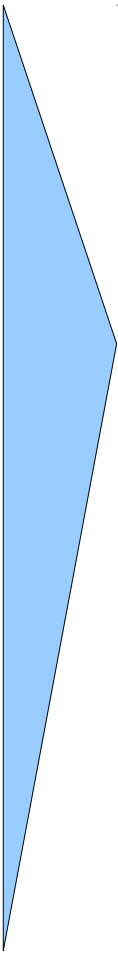
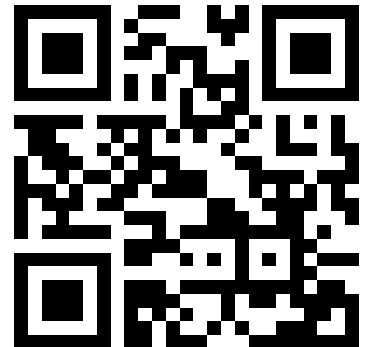


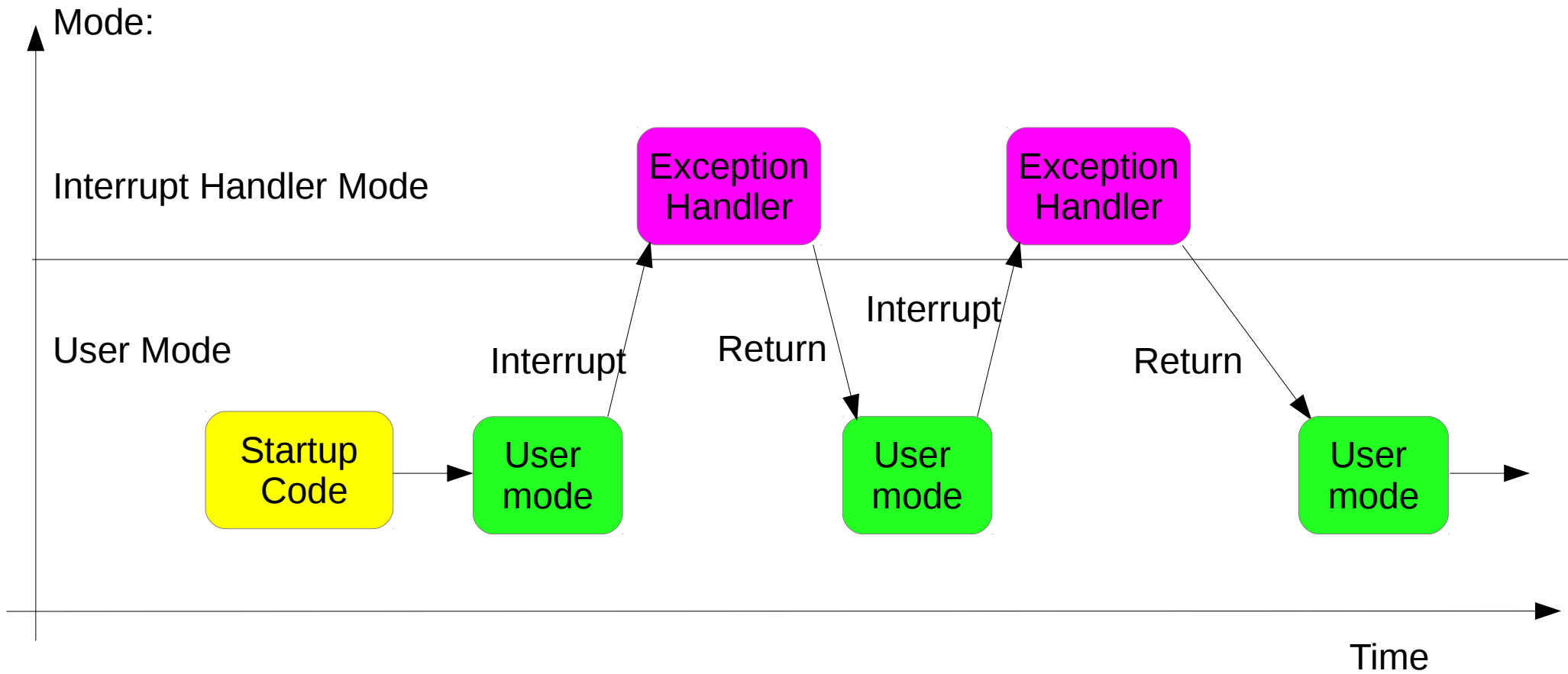
# Lecture Overview

- 
- Introduction, Microcontroller basics
  - 32-bit Microcontroller technology
  - ARM Cortex M Architecture, (CM0 CM3 CM4 CM7)
  - Cortex M Microcontroller Instruction Set
  - Exception Handling, Interrupts
  - Real-time Operating Systems (RTOS)
  - Memory Management, protected memory, MPU
  - Implementing Calculations, Data Formats, FPU
  - I/O, Device drivers
  - Application Design
  - Case Studies, Advanced Design Patterns

***<https://skript.eit.h-da.de/ams/>***



# Interrupt



# Exceptions and Interrupt

Fast response to external events

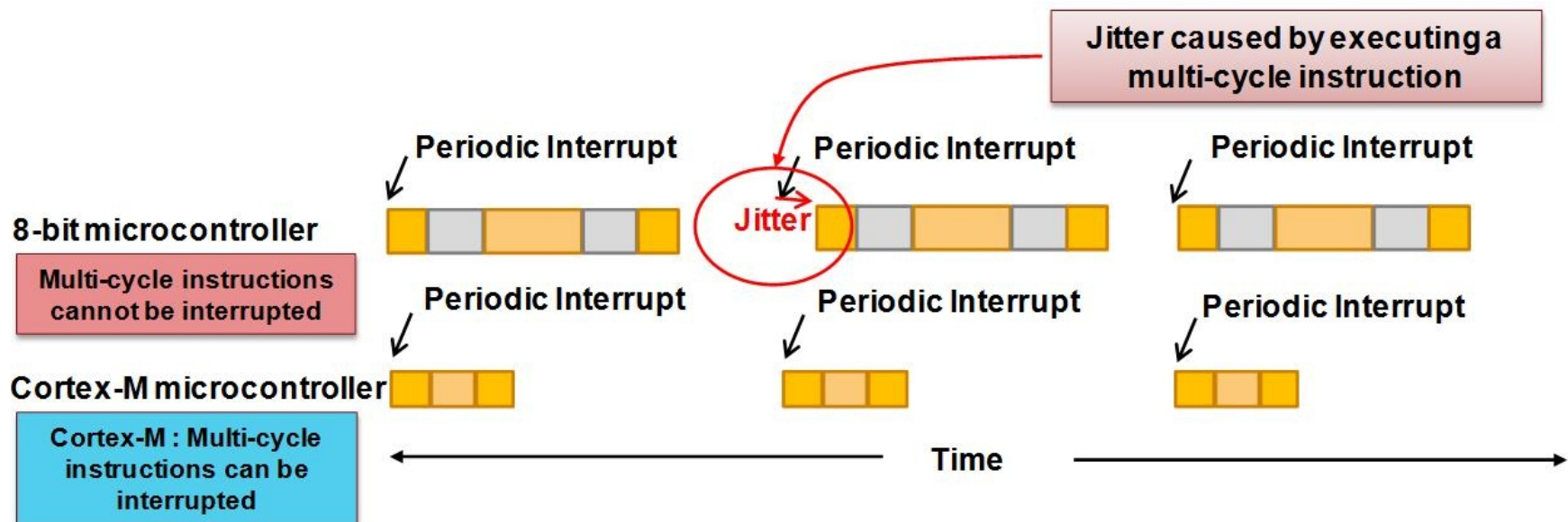
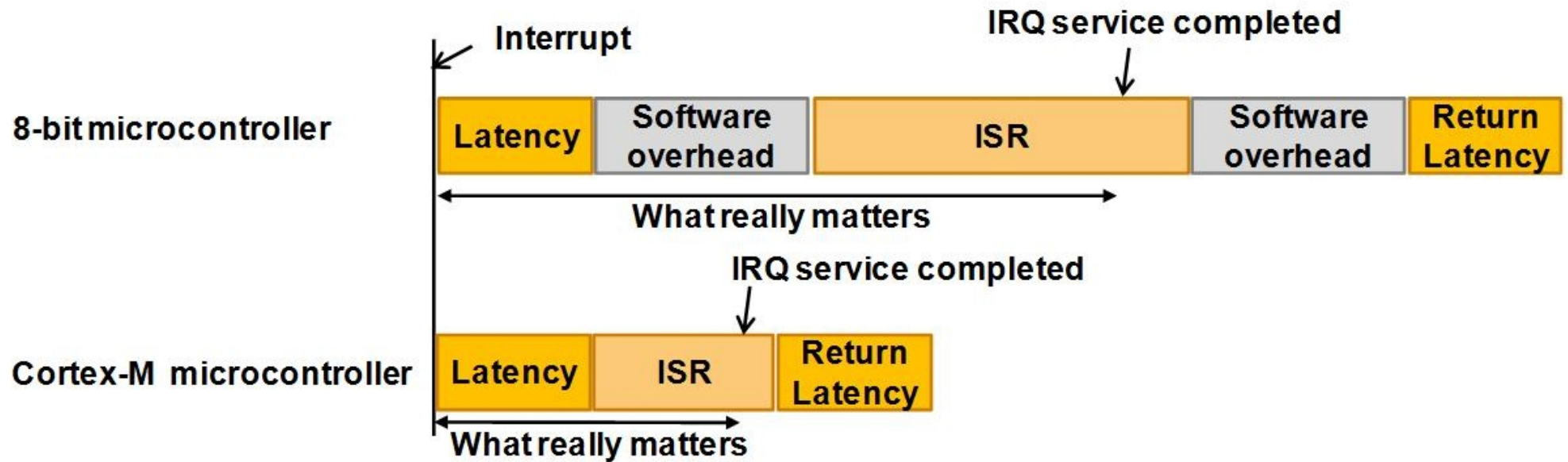
Handling of exception-situations

Cortex architecture has  
**Exceptions and Interrupts**

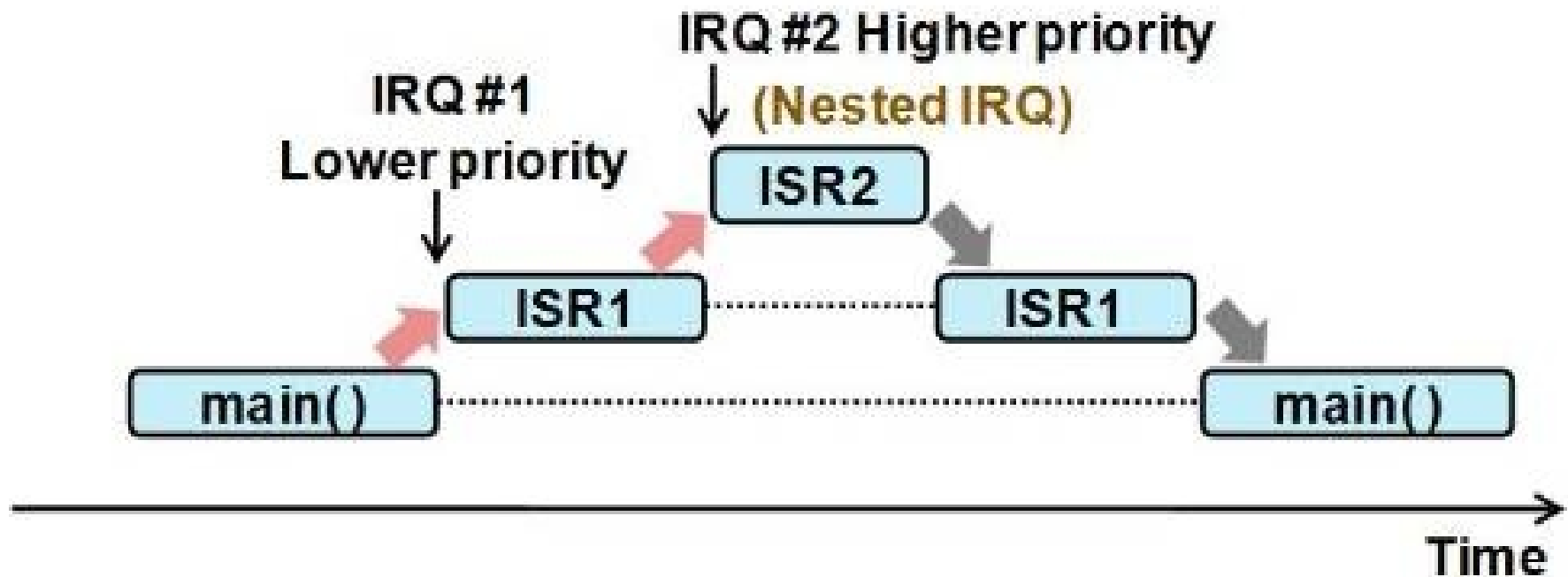
CM3: 1-240 external interrupts (**IRQs**)

Nested Vectored Interrupt Controller  
**NVIC** (0xe000e000...)

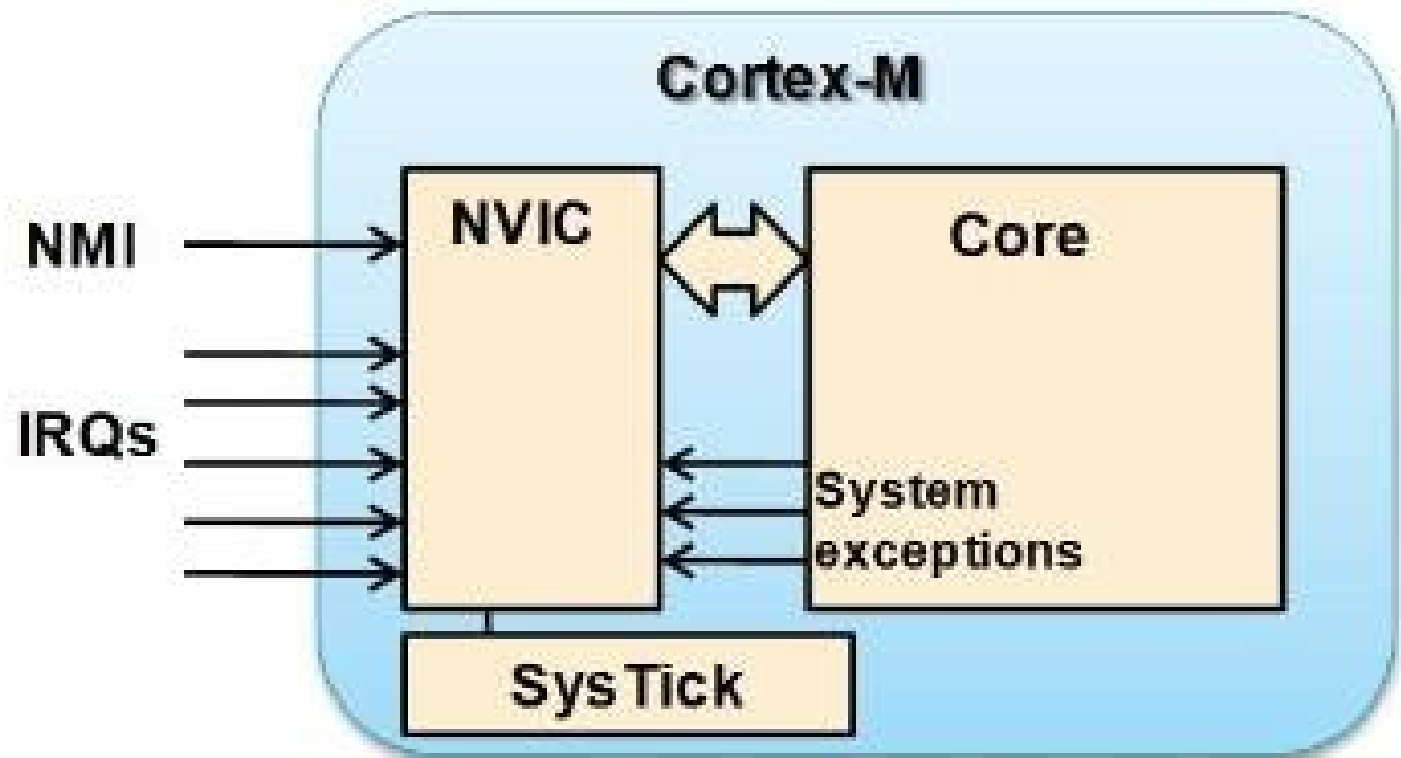
# Interrupt Latency and Jitter



# Nested Interrupts



# Cortex M Interrupt System

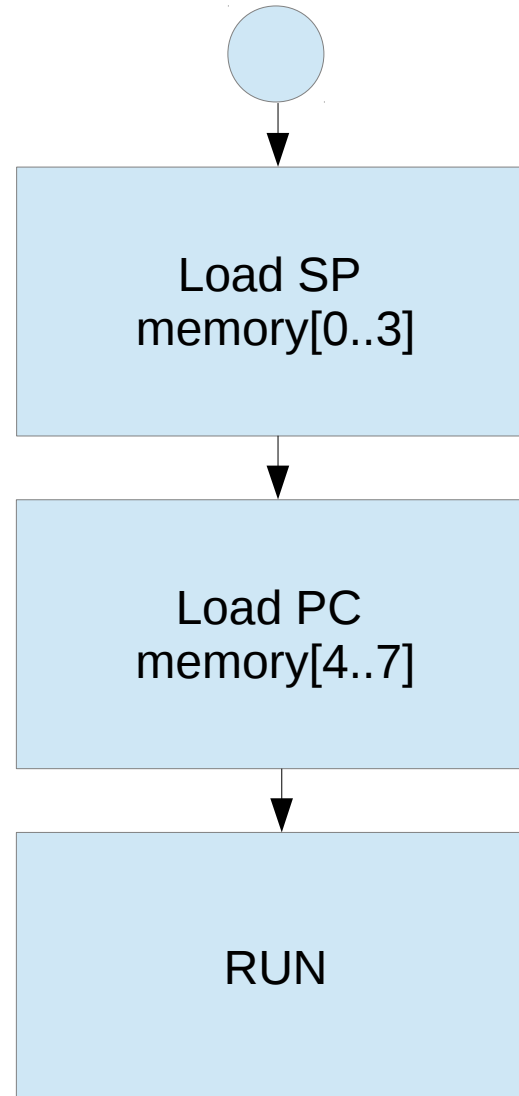


# Cortex-M3 Exception Handling

---

- **Reset** : power-on or system reset
- **NMI** : cannot be stopped or preempted by any exception other than reset
- **Faults**
  - **Hard Fault** : default Fault or any fault unable to activate
  - **Memory Manage** : MPU violations
  - **Bus Fault** : prefetch and memory access violations
  - **Usage Fault** : undef instructions, divide by zero, etc.
- **SVC**all : privileged OS requests
- **Debug Monitor** : debug monitor program
- **PendSV** : pending SVCalls
- **SysTick Interrupt** : internal sys timer, i.e., used by RTOS to periodically check resources or peripherals
- **External Interrupt** : i.e., external peripherals

# Reset Sequence



Normal SP setting: Directly **above** last RAM cell !

Vector addresses need to be odd (LSB=1) to indicate THUMB state !

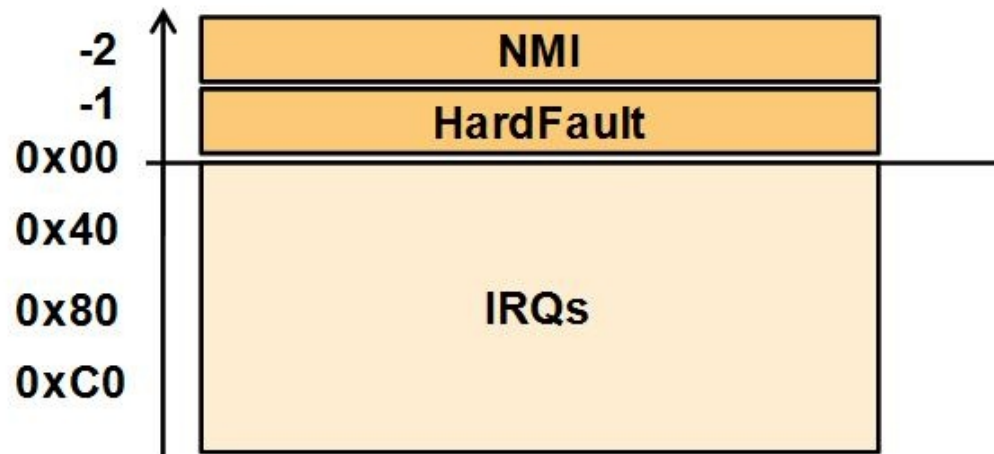


# Exceptions and Interrupts

|      | Exception Number | Exception Type  | Priority (Default to 0 if Programmable) | Description                                                             |
|------|------------------|-----------------|-----------------------------------------|-------------------------------------------------------------------------|
| CORE | 0                | NA              | NA                                      | No exception running                                                    |
|      | 1                | Reset           | -3 (Highest)                            | Reset                                                                   |
|      | 2                | NMI             | -2                                      | NMI (external NMI input)                                                |
|      | 3                | Hard fault      | -1                                      | All fault conditions, if the corresponding fault handler is not enabled |
|      | 4                | MemManage fault | Programmable                            | Memory management fault; MPU violation or access to illegal locations   |
|      | 5                | Bus fault       | Programmable                            | Bus error (prefetch abort or data abort)                                |
|      | 6                | Usage fault     | Programmable                            | Program error                                                           |
| RTOS | 7-10             | Reserved        | NA                                      | Reserved                                                                |
|      | 11               | SVCall          | Programmable                            | Supervisor call                                                         |
|      | 12               | Debug monitor   | Programmable                            | Debug monitor (break points, watchpoints, or external debug request)    |
|      | 13               | Reserved        | NA                                      | Reserved                                                                |
|      | 14               | PendSV          | Programmable                            | Pendable request for system service                                     |
|      | 15               | SYSTICK         | Programmable                            | System tick timer                                                       |
| I/O  | 16               | IRQ #0          | Programmable                            | External interrupt #0                                                   |
|      | 17               | IRQ #1          | Programmable                            | External interrupt #1                                                   |
|      | ...              | ...             | ...                                     | ...                                                                     |
|      | 255              | IRQ #239        | Programmable                            | External interrupt #239                                                 |

# Interrupt Priority

Higher priority



Priority level registers  
in Cortex-M0/M0+



Priority level registers  
in Cortex-M3/M4



# Interrupt Vector Table

```
g_pfnVectors:  
| .word _estack  
| .word Reset_Handler  
| .word NMI_Handler  
| .word HardFault_Handler  
| .word MemManage_Handler  
| .word BusFault_Handler  
| .word UsageFault_Handler  
| .word 0  
| .word 0  
| .word 0  
| .word 0  
| .word vPortSVCHandler  
| .word DebugMon_Handler  
| .word 0  
| .word xPortPendSVHandler  
| .word xPortSysTickHandler  
| .word WWDG_IRQHandler  
| .word PVD_IRQHandler  
| .word TAMPER_IRQHandler  
| .word RTC_IRQHandler  
| .word FLASH_IRQHandler  
| .word RCC_IRQHandler  
| .word EXTI0_IRQHandler  
| .word EXTI1_IRQHandler  
| .word EXTI2_IRQHandler  
| .word EXTI3_IRQHandler  
| .word EXTI4_IRQHandler  
| .word DMA1_Channel1_IRQHandler  
| .word DMA1_Channel2_IRQHandler  
| .word DMA1_Channel3_IRQHandler
```

This table is usually located at address 0x00000000 ,  
but it can be relocated by writing the **NVIC\_VECTTBL** register

# Two Stackpointers !

Control-Register CONTROL (1 bit)

CONTROL = 0 → Only MSP is used in thread and handler mode

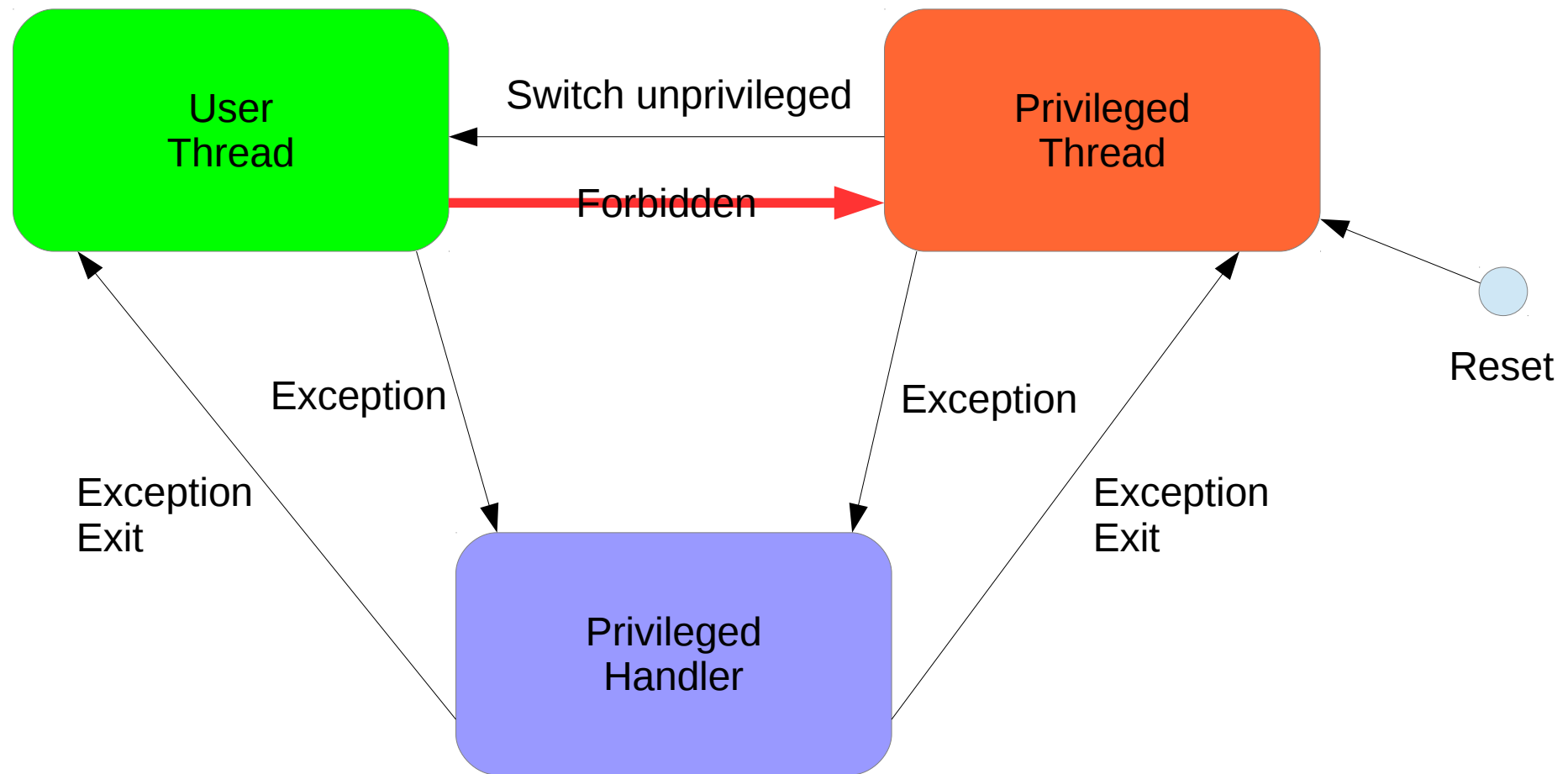
CONTROL = 1 → PSP is used only in thread mode  
→ MSP used within all exception handlers

| Name      | Functions (and banked registers)                                |
|-----------|-----------------------------------------------------------------|
| R0        | General-purpose register                                        |
| R1        | General-purpose register                                        |
| R2        | General-purpose register                                        |
| R3        | General-purpose register                                        |
| R4        | General-purpose register                                        |
| R5        | General-purpose register                                        |
| R6        | General-purpose register                                        |
| R7        | General-purpose register                                        |
| R8        | General-purpose register                                        |
| R9        | General-purpose register                                        |
| R10       | General-purpose register                                        |
| R11       | General-purpose register                                        |
| R12       | General-purpose register                                        |
| R13 (MSP) | R13 (PSP) Main Stack Pointer (MSP), Process Stack Pointer (PSP) |
| R14       | Link Register (LR)                                              |
| R15       | Program Counter (PC)                                            |

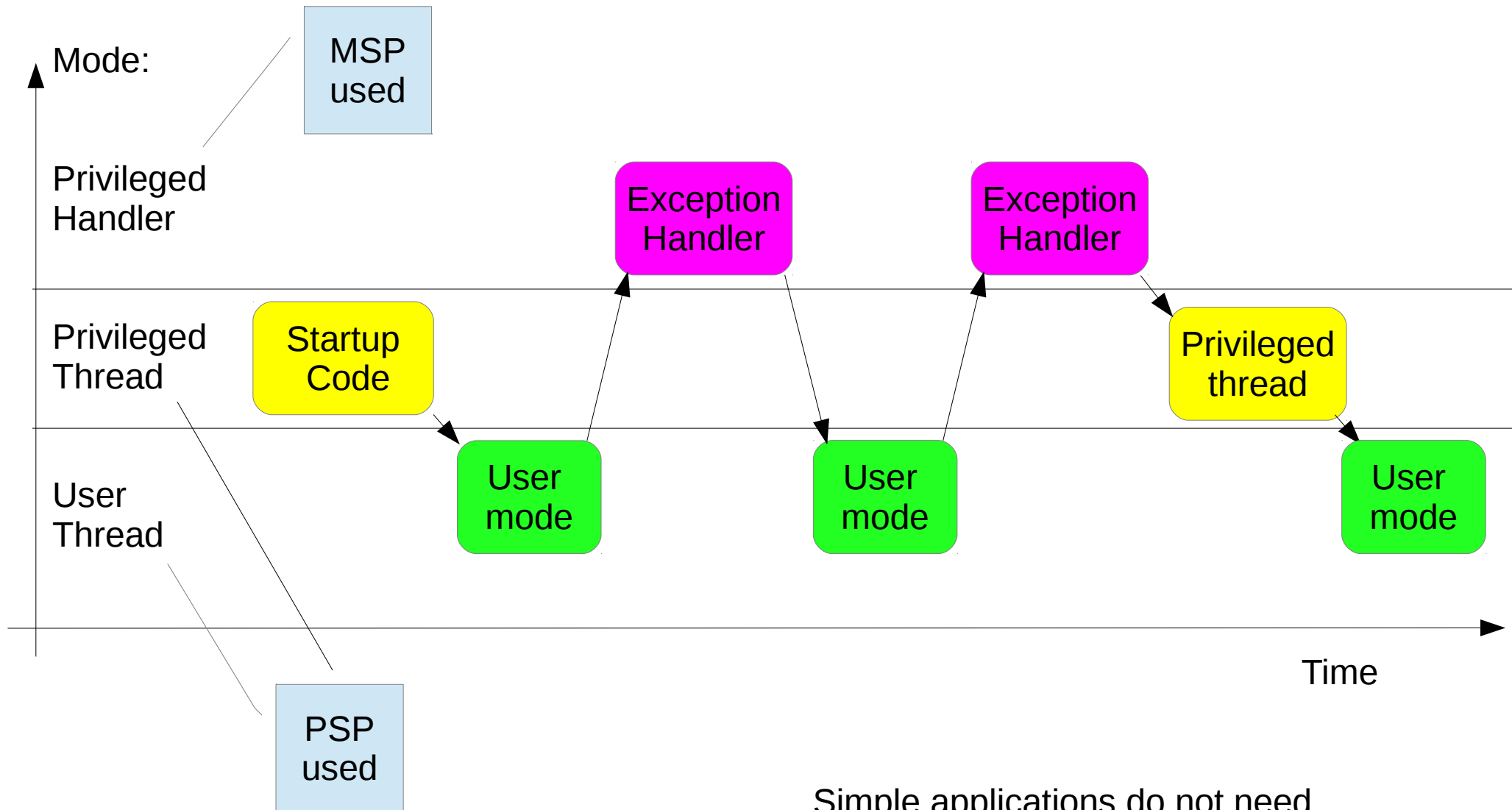
Low registers

High registers

# CortexM Operation Modes



# CM3 Operation Mode

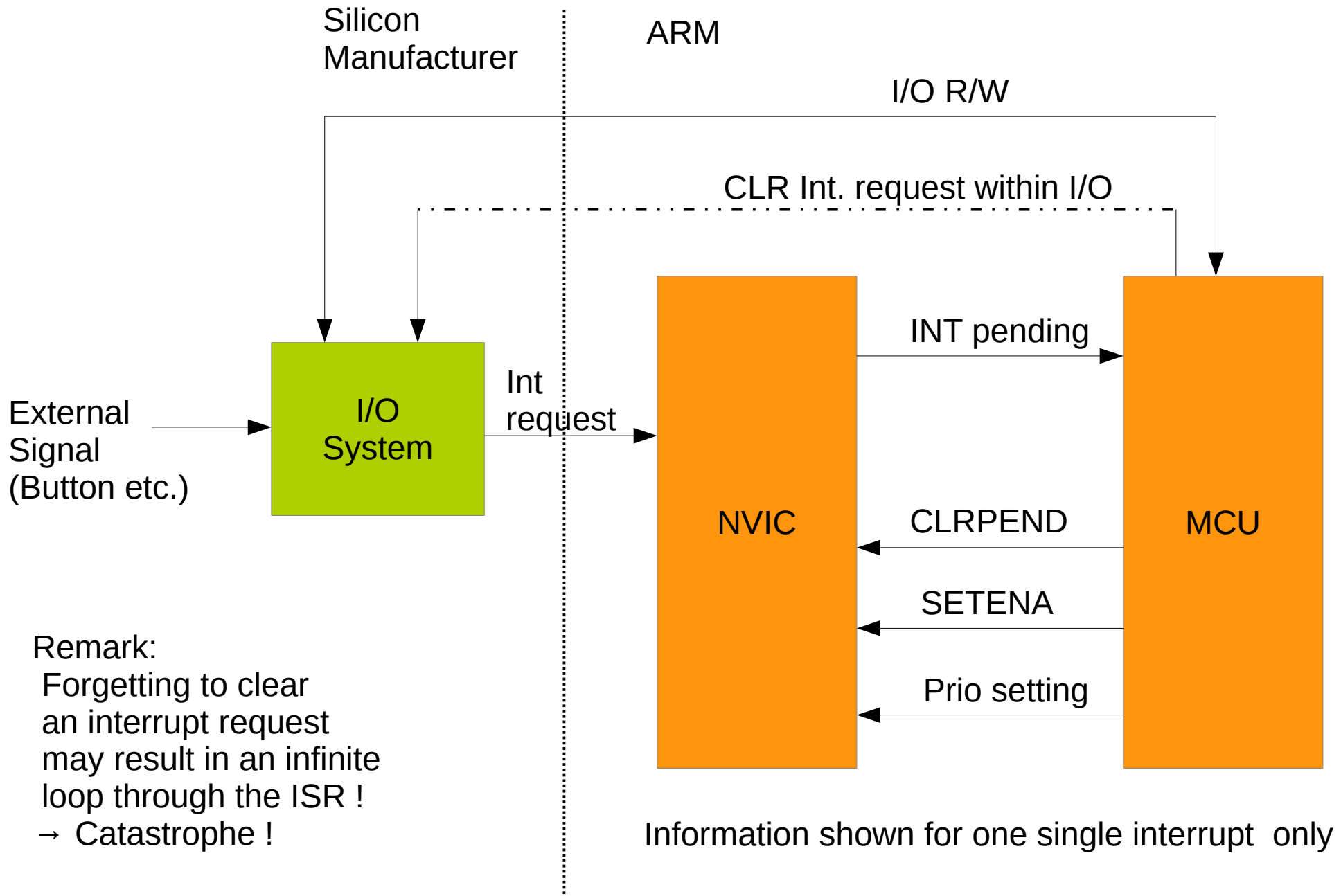


Simple applications do not need to use the user thread (unprivileged) mode

# Nested Vectored Interrupt Controller (NVIC)

- Low latency, low jitter interrupt response
- Interrupt service routines in pure C
- No need for assembly programming
- Interrupt service routines are just simple subroutines  
LR = 0xffffffff means “return from interrupt”
- The NVIC is accessed via memory-mapped I/O

# Interrupt Hardware





# NVIC Registers

SETENA0..2

Enable bits for external interrupts

CLRENA0..2

Disable bits for external interrupts

SETPEND0..2

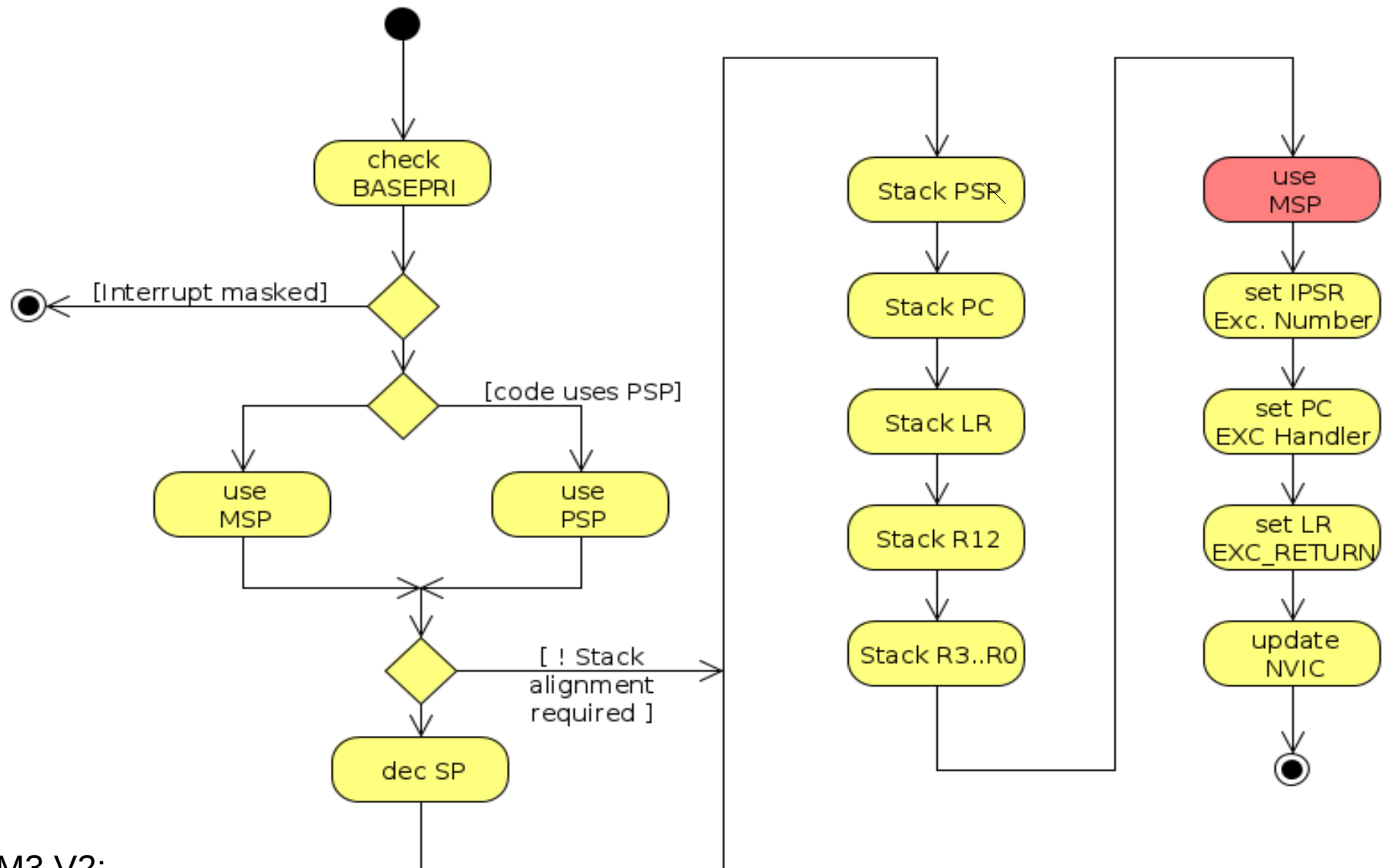
Set pending bits for external interrupts

CLRPEND0..2

Set pending bits for external interrupts

Write a “1” to clear a bit !

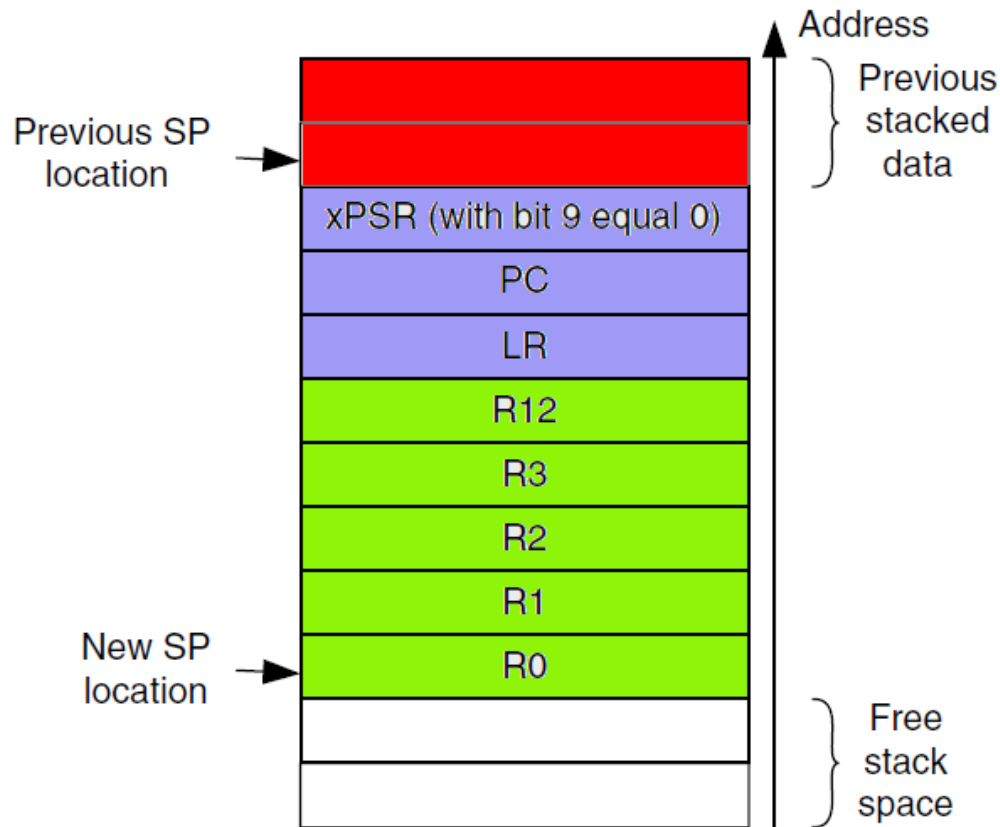
# Interrupt / Exception Sequence



CM3 V2:  
Stack frame aligned @ 64bits

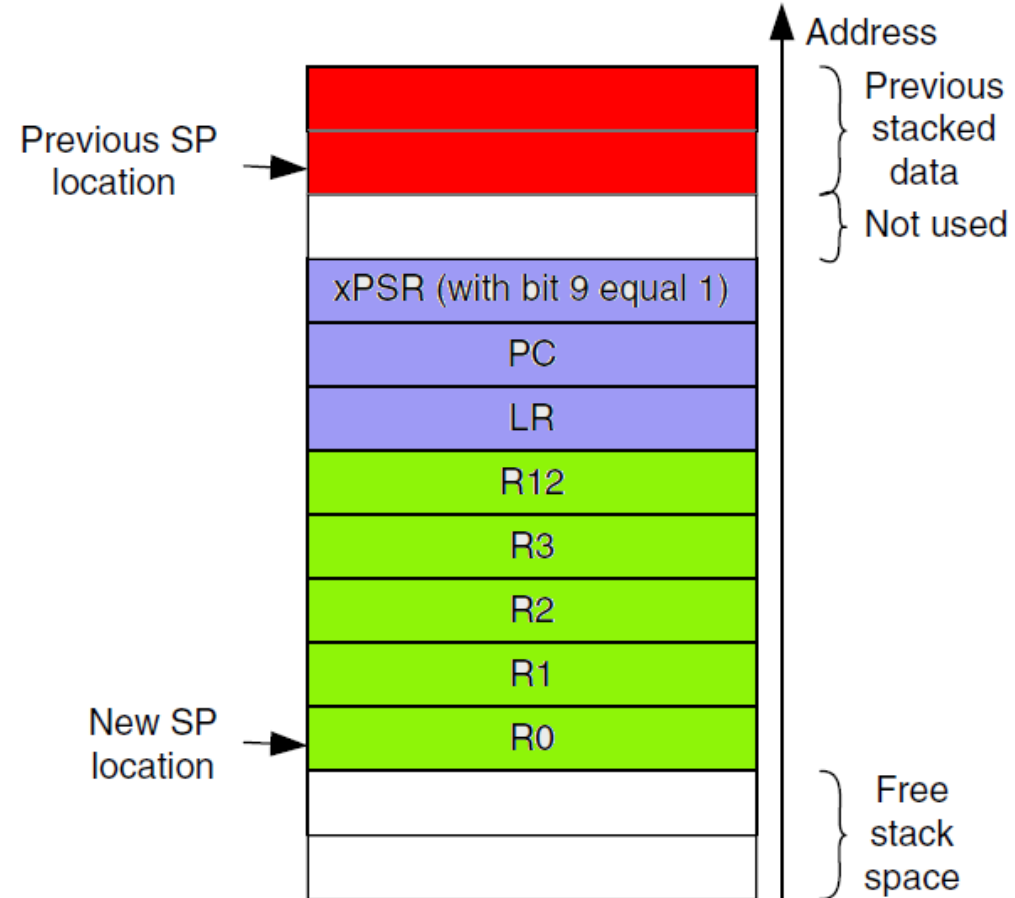
# Interrupt Stack Frame

Stack align adjustment not required



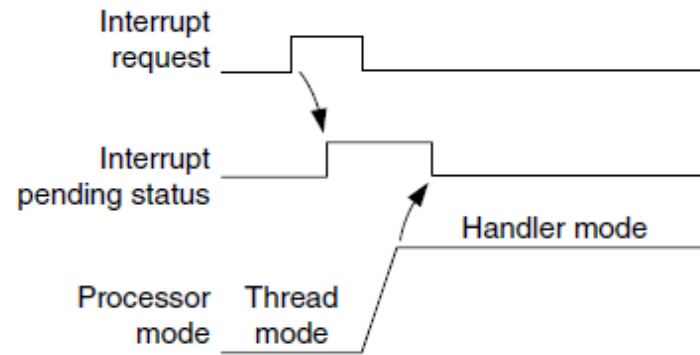
Previous stack point at double word address or STKALIGN is 0

Stack align adjustment required



Previous stack point not at double word address and STKALIGN is 1

# Pending Behavior

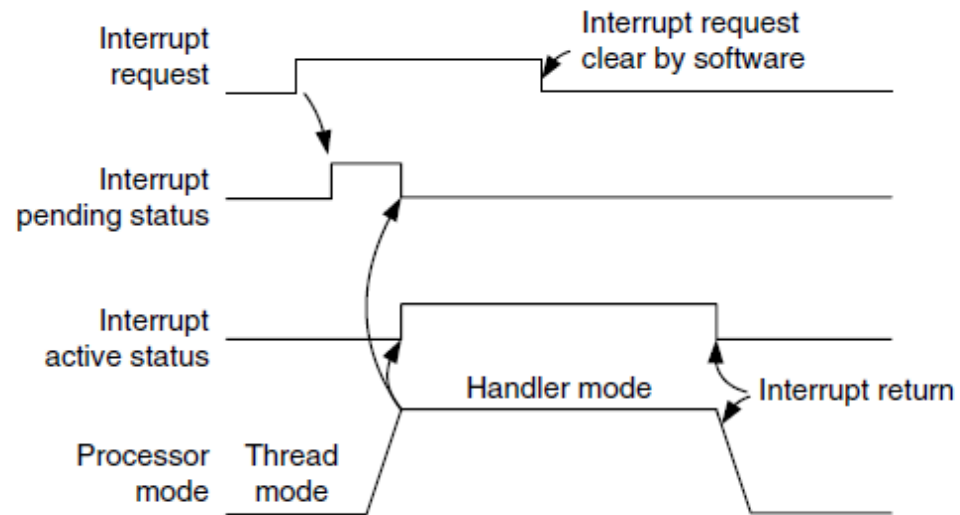


Interrupt request pulsed by I/O hardware.

Interrupt pending status automatically cleared within NVIC  
When interrupt handler is started.

No additional action to be taken within the ISR.

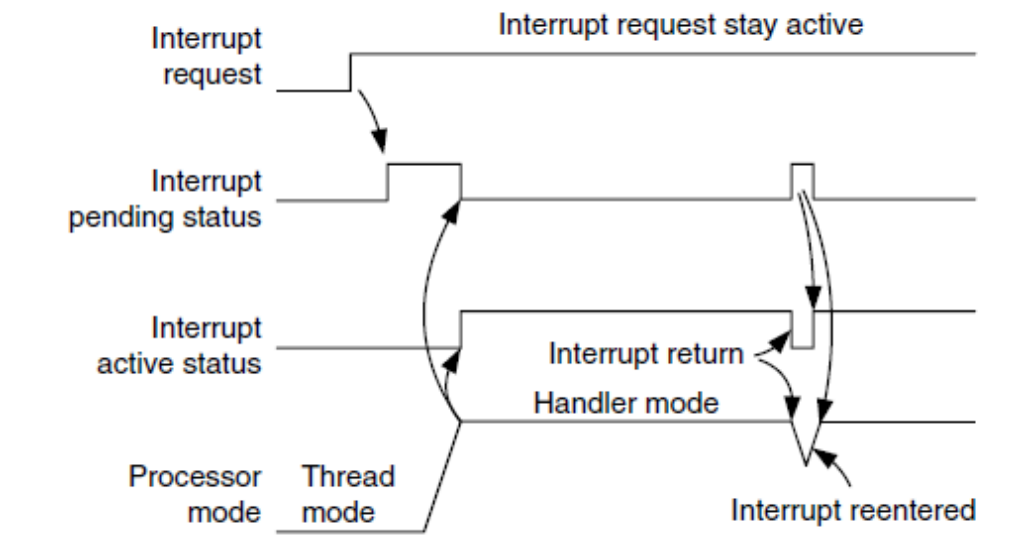
# Pending Behavior



Interrupt request cleared by software within the handler routine.  
Interrupt reason removed.

The status “interrupt pending” is cleared automatically  
on entry of the handler routine.

# Pending Behavior

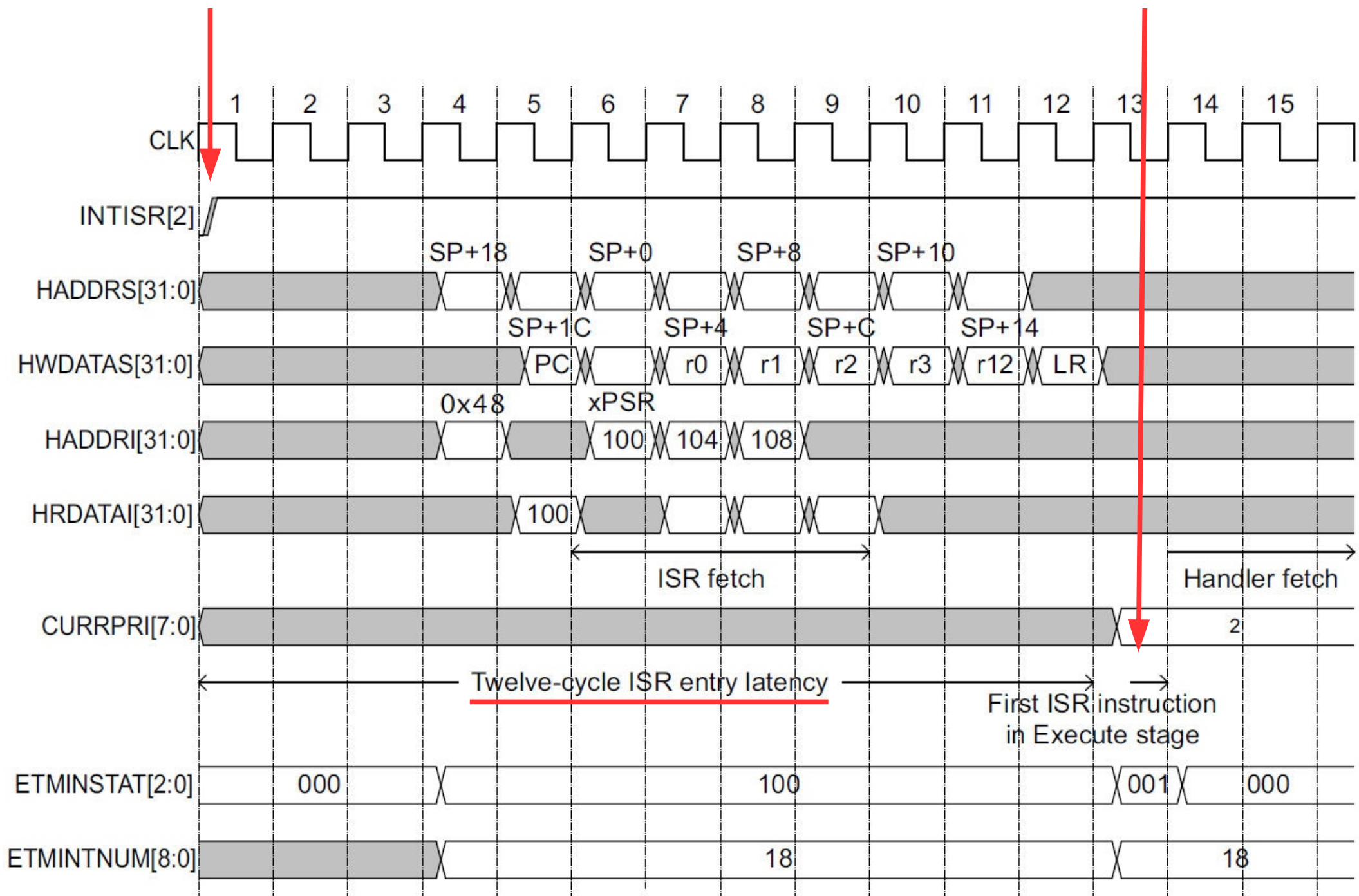


Catastrophe:

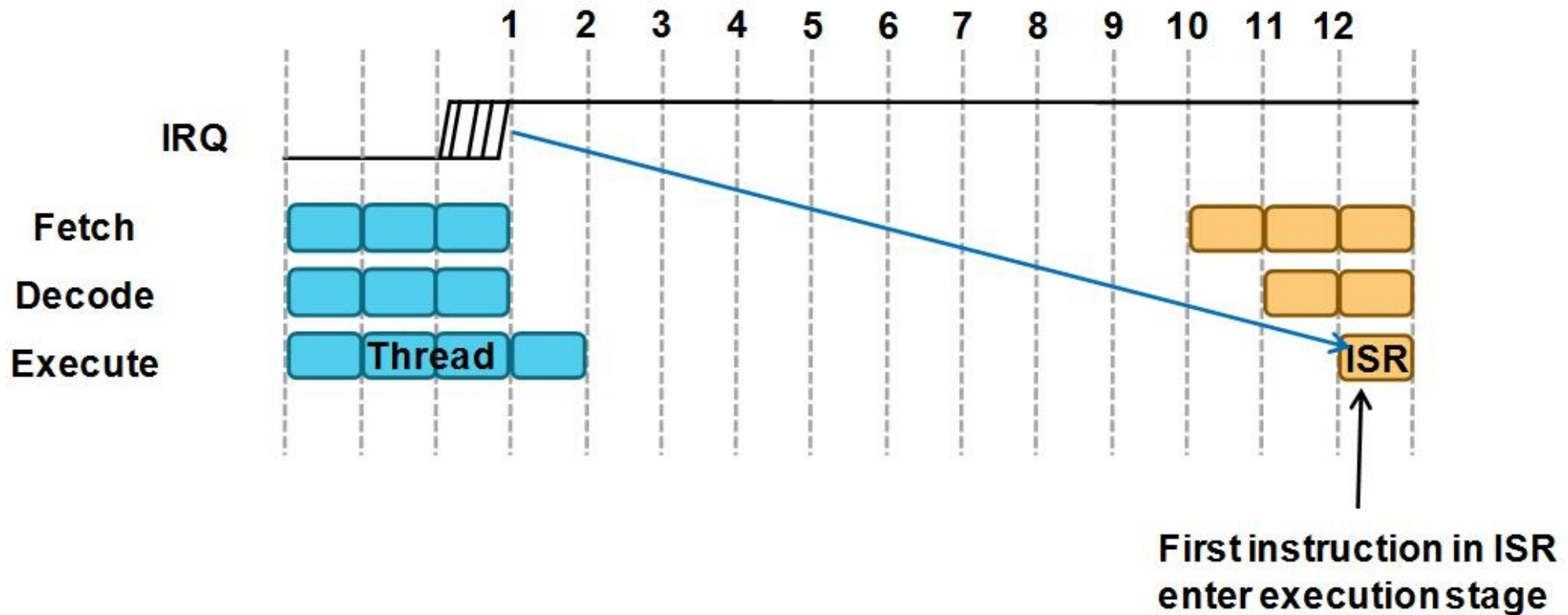
Interrupt reason not removed within handler

Interrupt request stays on forever

**System locked up completely !!!**



# Pipeline and ISR



The pipeline content is always wasted.  
Therefore a high interrupt rate slows the system down !



# Exception Return “Pseudo Address”

The return from an exception is done by loading a special value into the PC. This value is 0xffffffff, see table below:

Allowed EXC\_RETURN Values on Cortex-M3

| Value       | Condition                                                 |
|-------------|-----------------------------------------------------------|
| 0xFFFFFFFF1 | Return to handler mode                                    |
| 0xFFFFFFFF9 | Return to thread mode and on return use the main stack    |
| 0xFFFFFFFDD | Return to thread mode and on return use the process stack |

# Interrupt Latency

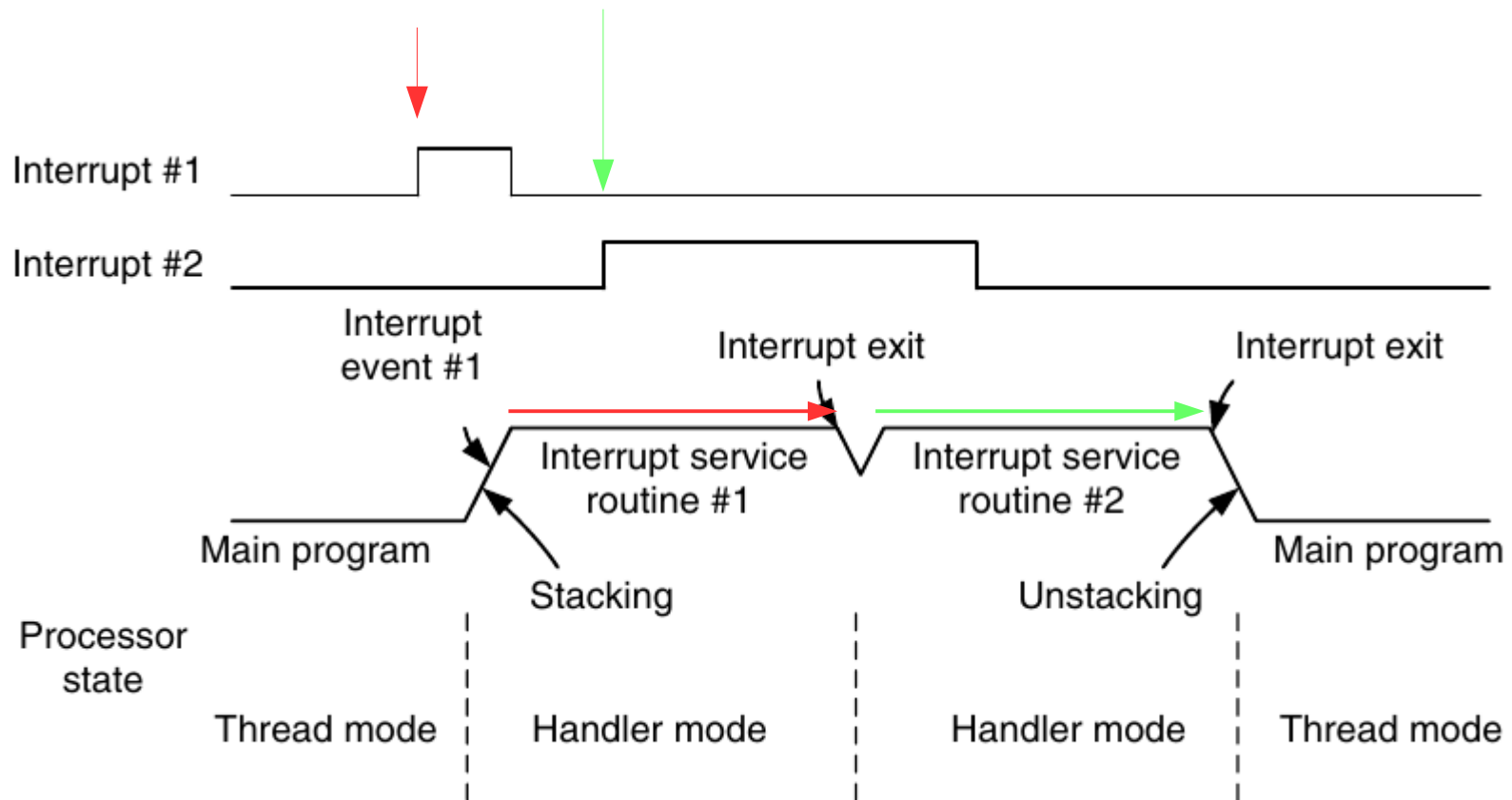
**The time between arrival of some interrupt and arrival inside of the ISR handler.**

CM3: Interrupt-latency is ideally **12 cycles** (up to **14 cycles** on a CM7)  
**Tail-chaining** takes only **6 cycles**  
(as register stacking is already done)

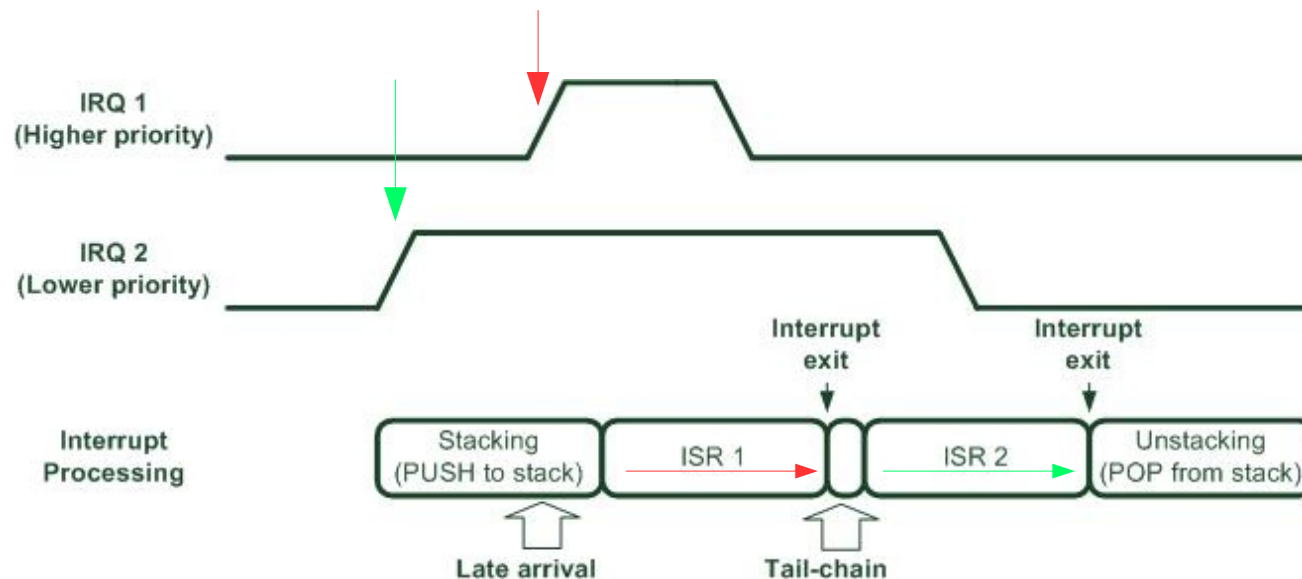
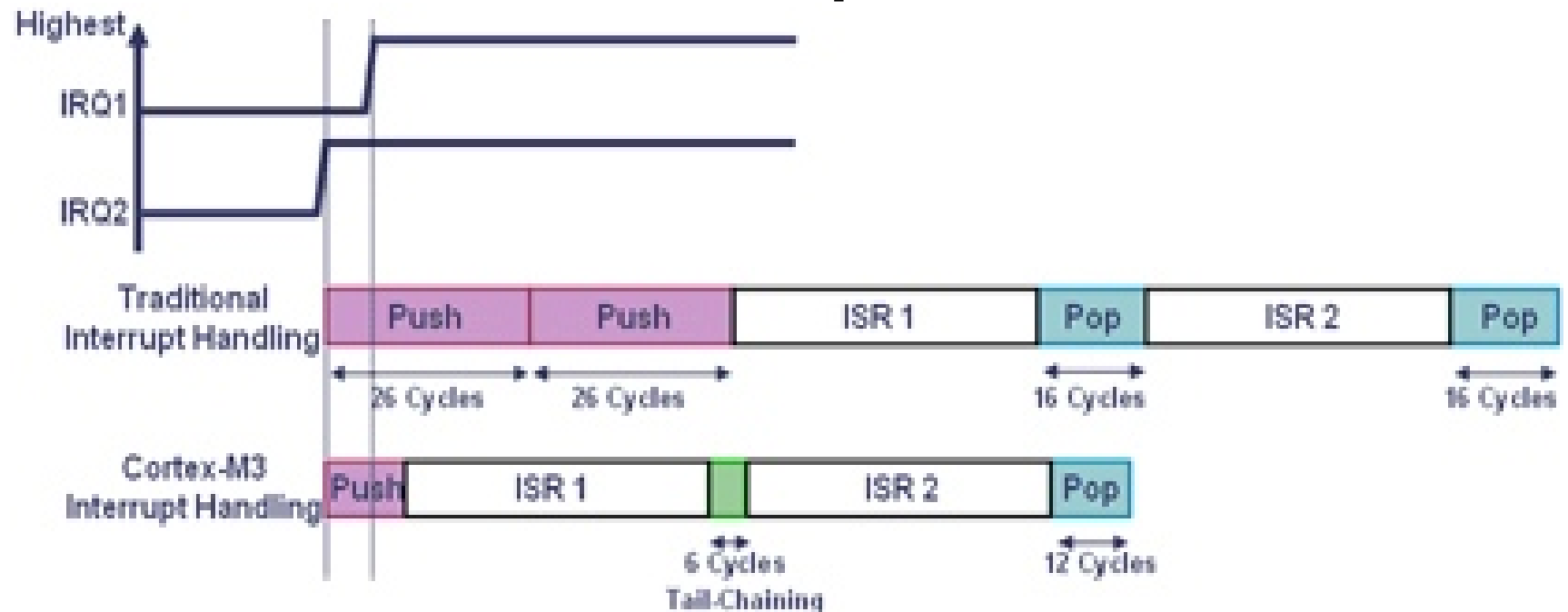
Occasionally instructions in progress are abandoned,  
example: ITTEE

Load and Store multiple instructions are frozen and continued after ISR return  
(Exception:  
Within ITTEE the LDM/STM is canceled and restarted on return from the ISR)

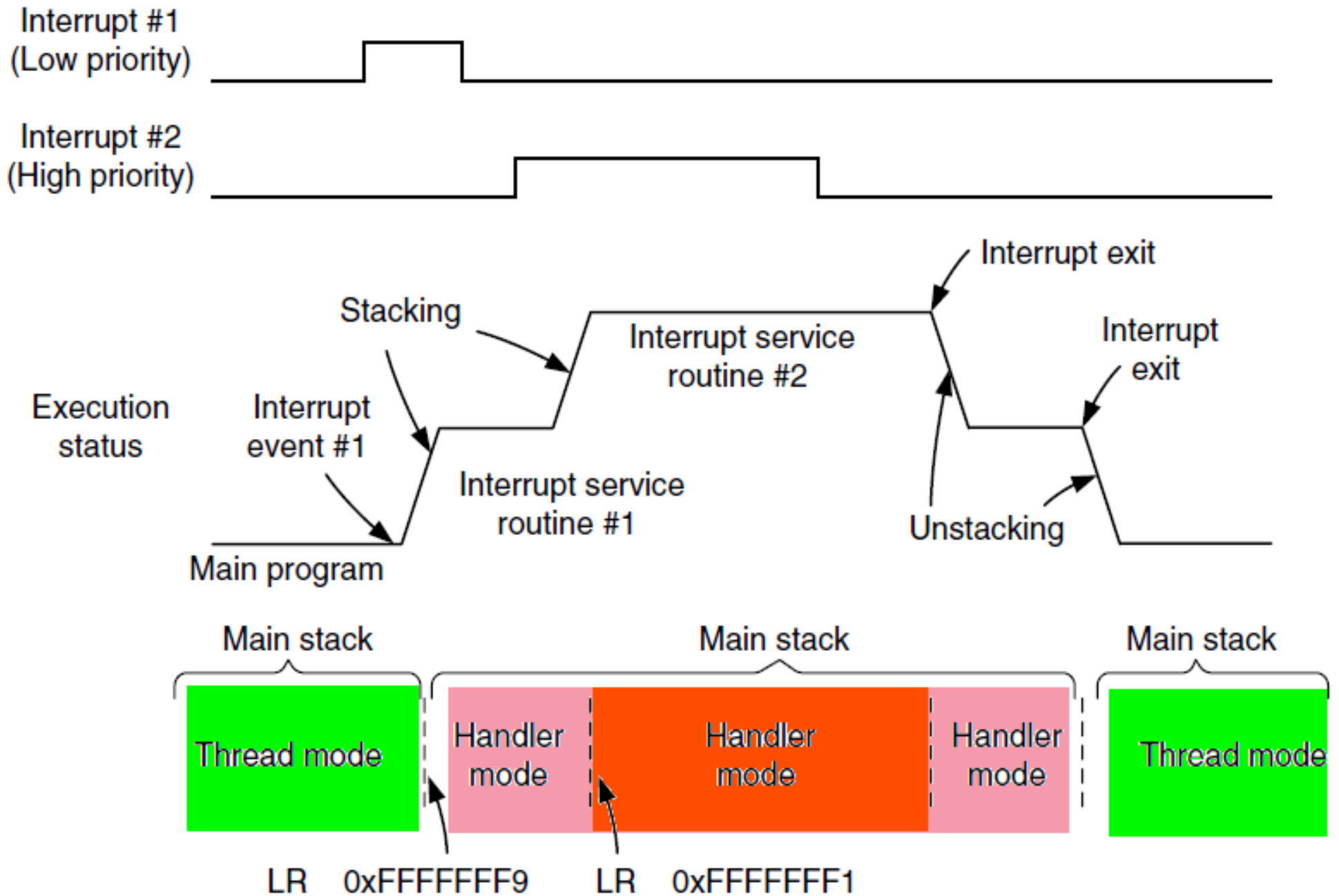
# Interrupt Tail-Chaining



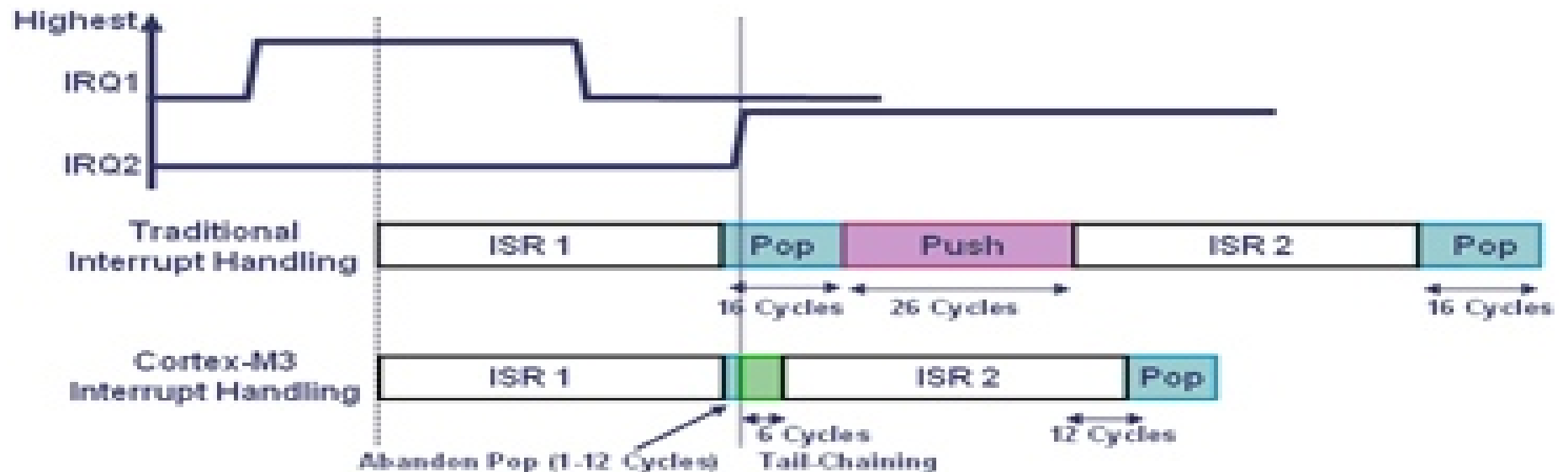
# Late Interrupt Arrival



# Nested Interrupts



# Stack Pop Preemption



# Enable / Disable Interrupts

PRIMASK Register:

```
void __enable_irq()  
__asm volatile ( "cpsie" )
```

```
mov     r0, #0  
msr     PRIMASK, r0
```

```
void __disable_irq()  
__asm volatile ( "cpsid" )
```

```
mov     r0, #1  
msr     PRIMASK, r0
```

# Rise / Lower Interrupt Priority

Write BASEPRI register to disable all interrupts above and including this priority number:

```
__set_BASEPRI( 0x60); // mask all interrupts above 0x5f
```

```
MOV    R0,#60  
MSR    BASEPRI,R0
```

```
__set_BASEPRI( 0);    // remove interrupt mask
```



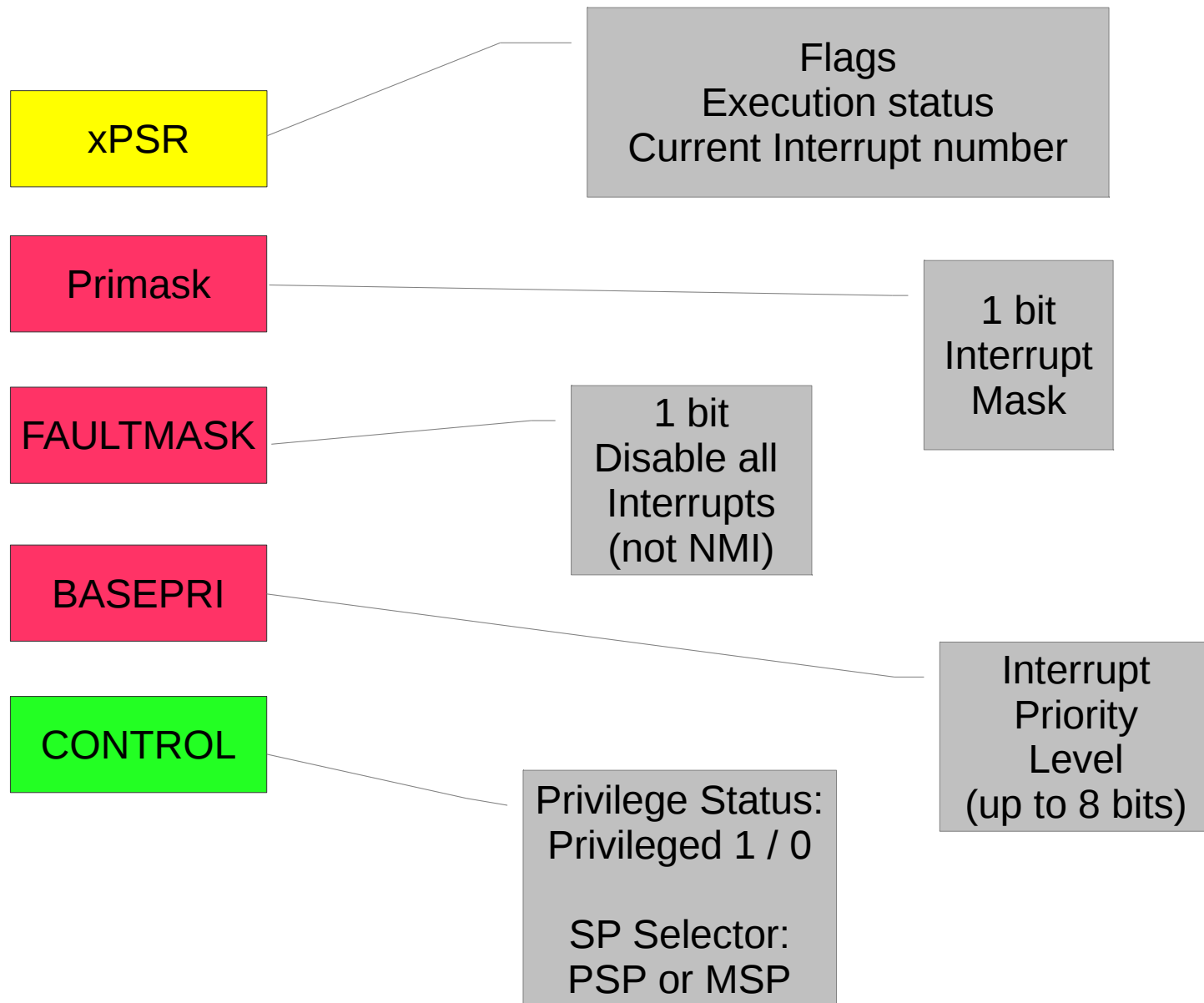
## Interrupt Control and State Register (0xE000ED04)

| Bits  | Name        | Type | Reset Value | Description                                                                                                                                  |
|-------|-------------|------|-------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| 31    | NMIPENDSET  | R/W  | 0           | NMI pended                                                                                                                                   |
| 28    | PENDSVSET   | R/W  | 0           | Write 1 to pend system call<br>Read value indicates pending status                                                                           |
| 27    | PENDSVCLR   | W    | 0           | Write 1 to clear PendSV pending status                                                                                                       |
| 26    | PENDSTSET   | R/W  | 0           | Write 1 to pend SYSTICK exception<br>Read value indicates pending status                                                                     |
| 25    | PENDSTCLR   | W    | 0           | Write 1 to clear SYSTICK pending status                                                                                                      |
| 23    | ISRPREEMPT  | R    | 0           | Indicates that a pending interrupt is going to be active in the next step (for debug)                                                        |
| 22    | ISRPENDING  | R    | 0           | External interrupt pending (excluding system exceptions such as NMI for fault)                                                               |
| 21:12 | VECTPENDING | R    | 0           | Pending ISR number                                                                                                                           |
| 11    | RETTOBASE   | R    | 0           | Set to 1 when the processor is running an exception handler; will return to thread level if interrupt return and no other exceptions pending |
| 9:0   | VECTACTIVE  | R    | 0           | Current running ISR                                                                                                                          |

## The System Handler Control and State Register (0xE000ED24)

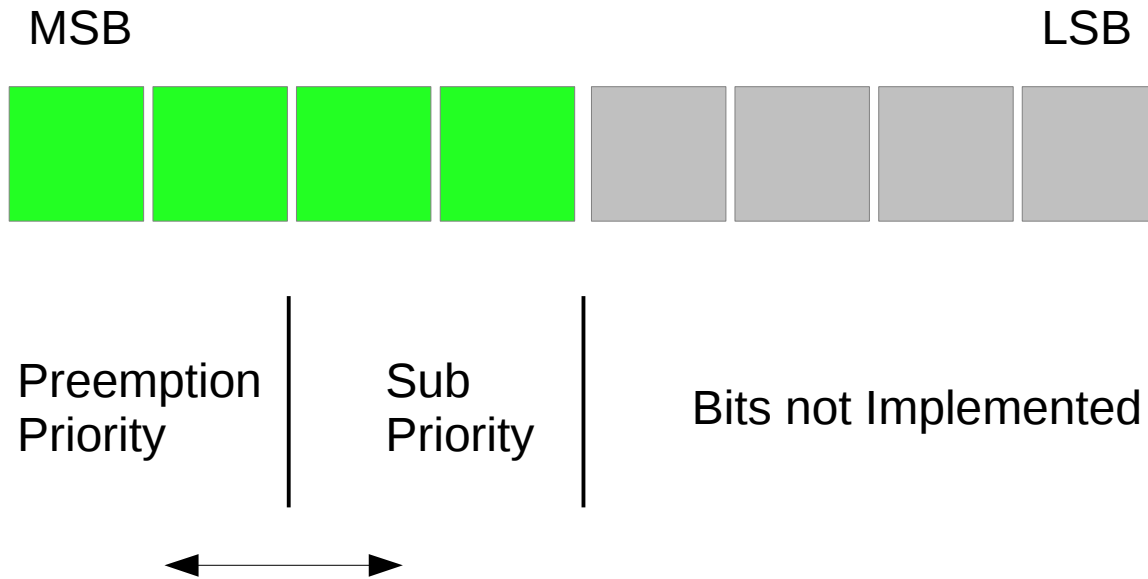
| Bits | Name           | Type | Reset Value | Description                                                                                                     |
|------|----------------|------|-------------|-----------------------------------------------------------------------------------------------------------------|
| 18   | USGFAULTENA    | R/W  | 0           | Usage fault handler enable                                                                                      |
| 17   | BUSFAULTENA    | R/W  | 0           | Bus fault handler enable                                                                                        |
| 16   | MEMFAULTENA    | R/W  | 0           | Memory management fault handler enable                                                                          |
| 15   | SVCALLPENDEd   | R/W  | 0           | SVC pended; SVC was started but was replaced by a higher-priority exception                                     |
| 14   | BUSFAULTPENDEd | R/W  | 0           | Bus fault pended; bus fault handler was started but was replaced by a higher-priority exception                 |
| 13   | MEMFAULTPENDEd | R/W  | 0           | Memory management fault pended; memory management fault started but was replaced by a higher-priority exception |
| 12   | USGFAULTPENDEd | R/W  | 0           | Usage fault pended; usage fault started but was replaced by a higher-priority exception                         |
| 11   | SYSTICKACT     | R/W  | 0           | Read as 1 if SYSTICK exception is active                                                                        |
| 10   | PENDSVACT      | R/W  | 0           | Read as 1 if PendSV exception is active                                                                         |
| 8    | MONITORACT     | R/W  | 0           | Read as 1 if debug monitor exception is active                                                                  |
| 7    | SVCALLACT      | R/W  | 0           | Read as 1 if SVC exception is active                                                                            |
| 3    | USGFAULTACT    | R/W  | 0           | Read as 1 if usage fault exception is active                                                                    |
| 1    | BUSFAULTACT    | R/W  | 0           | Read as 1 if bus fault exception is active                                                                      |
| 0    | MEMFAULTACT    | R/W  | 0           | Read as 1 if memory management fault is active                                                                  |

# Special “Registers”



# Priority Setting

Individually programmable for almost all interrupts



Preemption/ Subpriority  
Width configurable

AIRCR – Register bits 10:8 (Priority Group setting)

Fixed Priority:

- 3 Reset
- 2 NMI
- 1 Hard Fault

# Preemption Priority

## Sub - Priority

Exceptions that share the same Preemption-Priority can NOT preempt each others.

If two such interrupts occur at the same time the one with the higher Sub-Priority will be serviced first.

Interrupts with higher Preemption-Priority can preempt lower priority interrupts.

The current interrupt priority can be read using the IPSR.

Attention: Lower number means “higher priority”!

# Supervisor Call SVC

Software triggered exception (“Software Interrupt”)

Automatically switches from user to handler mode

Used for calls to the RTOS

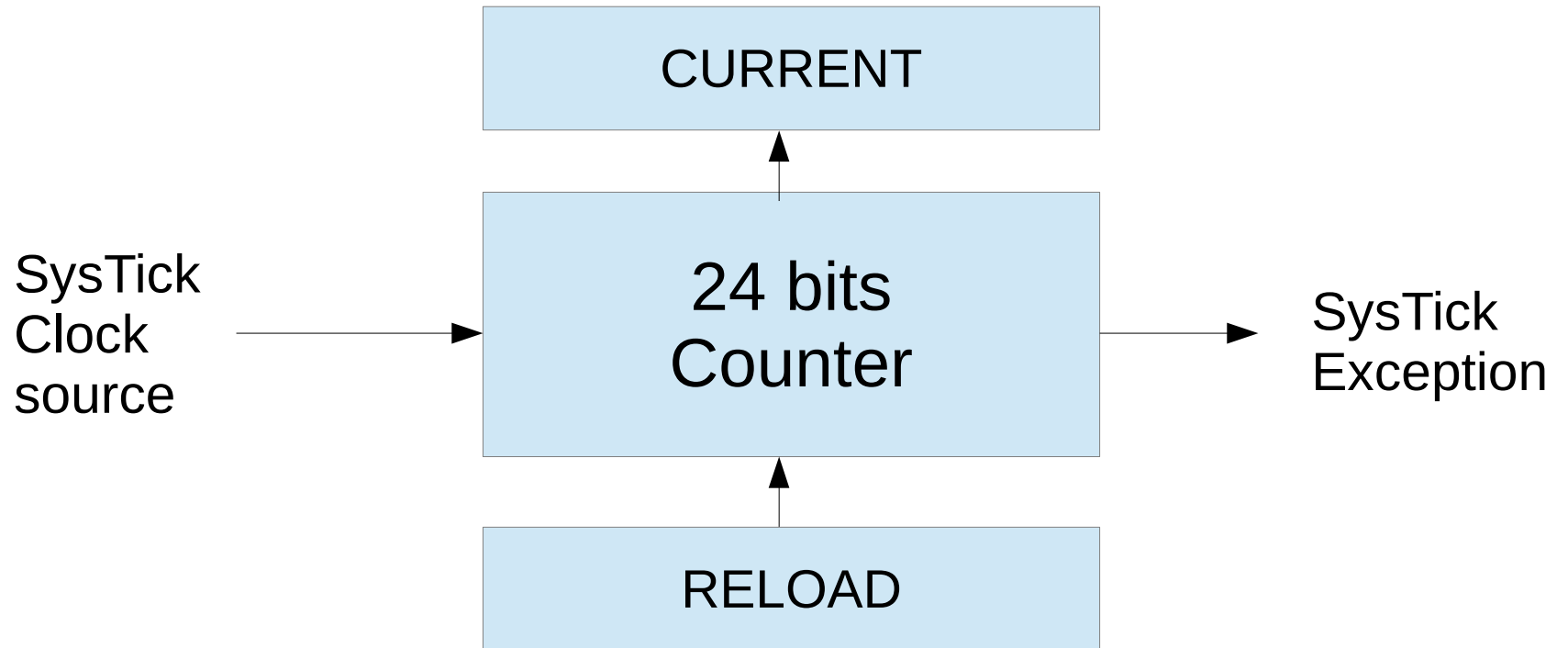
Also used to rise the priority / privilege mode

Example:

```
__asm( “SVC    13”);
```

The number needs to be picked from the program memory via the return stack as it is not propagated to the ISR routine.

# Systick Timer



# Systick Timer Registers

| 1 SYSTICK Control and Status Register (0xE000E010) |           |      |             |                                                                                                                                                   |
|----------------------------------------------------|-----------|------|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Bits                                               | Name      | Type | Reset Value | Description                                                                                                                                       |
| 16                                                 | COUNTFLAG | R    | 0           | Read as 1 if counter reaches 0 since last time this register is read; clear to 0 automatically when read or when current counter value is cleared |
| 2                                                  | CLKSOURCE | R/W  | 0           | 0 = External reference clock (STCLK)<br>1 = Use processor free running clock                                                                      |
| 1                                                  | TICKINT   | R/W  | 0           | 1 = Enable SYSTICK interrupt generation when SYSTICK Timer reaches 0<br>0 = Do not generate interrupt                                             |
| 0                                                  | ENABLE    | R/W  | 0           | SYSTICK Timer enable                                                                                                                              |

| SYSTICK Reload Value Register (0xE000E014) |        |      |             |                                   |
|--------------------------------------------|--------|------|-------------|-----------------------------------|
| Bits                                       | Name   | Type | Reset Value | Description                       |
| 23:0                                       | RELOAD | R/W  | 0           | Reload value when timer reaches 0 |

| SYSTICK Current Value Register (0xE000E018) |         |      |             |                                                                                                                                                                   |
|---------------------------------------------|---------|------|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Bits                                        | Name    | Type | Reset Value | Description                                                                                                                                                       |
| 23:0                                        | CURRENT | R/Wc | 0           | Read to return current value of the timer.<br>Write to clear counter to 0. Clearing of current value also clears COUNTFLAG in SYSTICK Control and Status register |



# Setup SysTick

## CMSIS Setup Function

```
__STATIC_INLINE uint32_t SysTick_Config(uint32_t ticks)
{
    if (ticks > SysTick_LOAD_RELOAD_Msk) return (1);

    SysTick->LOAD = (ticks & SysTick_LOAD_RELOAD_Msk) - 1;
    NVIC_SetPriority(SysTick_IRQn, (1<< NVIC_PRIO_BITS) - 1);
    SysTick->VAL = 0;
    SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk |
                    SysTick_CTRL_TICKINT_Msk |
                    SysTick_CTRL_ENABLE_Msk;

    return (0);
}
```

# Get Time in Microseconds

```
extern "C" uint64_t getTime_usec(void) // read time from hard + software
{
    uint64_t time;
    uint64_t present_systick;
    uint64_t reload = (* (uint32_t *)0xe000e014) + 1;

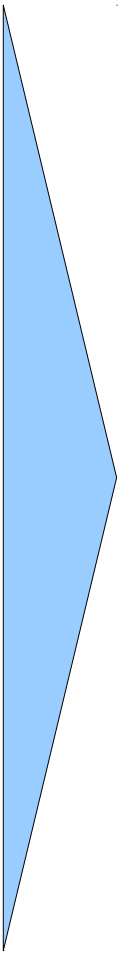
    do
    {
        __DSB();
        time= __LDREXW( (uint32_t*)&SystemTicks);
        present_systick=(uint64_t) ( * (uint32_t *)0xe000e018);
        __DSB();
    }
    while( __STREXW( (uint32_t)time, (uint32_t*)&SystemTicks) != 0)

    ; // milliseconds -> microseconds
    time *= 1000;

    present_systick = reload - present_systick; // because it's a down-counter
    present_systick *= 1000; // millisecs -> microsecs
    present_systick /= reload;

    return time + present_systick;
}
```

# Lecture Overview

- 
- Introduction, Microcontroller basics
  - 32-bit Microcontroller technology
  - ARM Cortex M Architecture, (CM0 CM3 CM4 CM7)
  - Cortex M Microcontroller Instruction Set
  - Exception Handling, Interrupts
  - Real-time Operating Systems (RTOS)
  - Memory Management, protected memory, MPU
  - Implementing Calculations, Data Formats, FPU
  - I/O, Device drivers
  - Application Design
  - Case Studies, Advanced Design Patterns

***<https://skript.eit.h-da.de/ams/>***

